Before C# 9, every .NET program had to include a `static void Main` function (it could also return `int`, `Task`, or `Task<int>`), typically declared in a class called `Program`. This function, which must exist, defines the entry point for your program. This code runs when you start your application, as in this example:

```
using System;
namespace MyApp
{
    public class Program
    {
        public static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

With top-level statements you can write the body of this method directly in the file, and the compiler generates the `Main` method for you.

When combined with C# 10 features such as implicit using statements, this dramatically simplifies the entry-point code of your app to

```
Console.WriteLine("Hello World!");
```

When you use the explicit `Main` function you can access the command-line arguments provided when the app was run using the `args` parameter. With top-level statements the `args` variable is also available as a `string[]`, even though it's not declared explicitly. You could echo each argument provided by using

```
foreach(string arg in args)
{
    Console.WriteLine(arg);
}
```

In .NET 7 all the default templates use top-level statements, and I use them throughout this book. Most of the templates include an option to use the explicit `Main` function if you prefer (using the `--use-program-main` option if you're using the CLI). For more information on top-level statements and their limitations, see http://mng.bz/4DZa. If you decide to switch approaches later, you can always add or remove the `Main` function manually as required.

In .NET 7 ASP.NET Core applications the top-level statements build and run a `WebApplication` instance, as shown in the following listing, which shows